MongoDB

Day 4

The Aggregation Pipeline

Compute, don't just fetch

What you'll master:

- sort · limit · skip · projection
- Pipeline concept + aggregate()
- $match + $group
- $sum · $avg · $min · $max
- $project (rename) · $addFields
- $cond (if / else grading)
- $out (save results)

Refining find() Results

sort · limit · skip · projection

`.sort({ marks: -1 })`	Order results. 1 = ascending, -1 = descending. Multi-field: { city:1, marks:-1 }
`.limit(3)`	Return at most n documents
`.skip(5)`	Ignore the first n documents
`find({}, { name:1, _id:0 })`	Projection: 1 include, 0 exclude

skip + limit together = pagination (page 2 of 5: .skip(5).limit(5))

The Aggregation Pipeline

Documents flow through ordered stages — each output feeds the next

Aggregation = process MANY documents -> return a combined / computed result (totals, averages, counts, groupings). Like SQL's GROUP BY / SUM / AVG.

each stage does ONE operation, then passes its output to the next

SYNTAX

```
db.<col>.aggregate([ {stage1}, {stage2}, ... ])
```

Runs documents through an ordered ARRAY of stages. Order matters.

```
db.students.aggregate([ { $match: {...} }, { $group: {...} } ])
```

$match — Filter Stage

Keep only documents that match — same syntax as a find() filter

SYNTAX

{ $match: { <conditions> } }

Filters documents. Uses all the Day 3 operators ($gt, $in, etc). Put it FIRST so later stages handle less data.

```
{ $match: { marks: { $gt: 80 } } }
```

```
// exact value
db.students.aggregate([
  { $match: { city: 'Delhi' } }
])

// -> Aarav, Neha, Rohan
```

```
// with an operator
db.students.aggregate([
  { $match:
      { marks: { $gt: 80 } }
  }
])
// -> 9 students above 80
```

$group — The Heart of Aggregation

Collapse many documents into groups, compute per group

SYNTAX

```
{ $group: { _id: '$field', newField: { <accumulator> } } }
```

_id is the GROUPING KEY (not the doc id!). Use '$field' with a $ prefix. Accumulators compute per group.

```
{ $group: { _id: '$city', totalStudents: { $sum: 1 } } }
```

```
db.students.aggregate([
  { $group: {
      _id: '$city',
      totalStudents: { $sum: 1 }
  } }
])
// { _id:'Delhi',  totalStudents:3 }
// { _id:'Mumbai', totalStudents:3 }
```

Watch out

`_id` = what to group BY, not the document id.

`$sum: 1` adds 1 per doc = a count.

`$sum: '$marks'` totals the marks.

$match + $group Together

Filter first, then group what's left

```
db.students.aggregate([
  // stage 1: keep only marks > 80
  { $match: { marks: { $gt: 80 } } },

  // stage 2: group survivors by city
  { $group: {
      _id: '$city',
      avgMarks: { $avg: '$marks' },
      totalStudents: { $sum: 1 }
  } }
])
```

Why order matters

$match BEFORE $group means only filtered docs get grouped — and it's faster (fewer docs reach the expensive $group). Delhi now shows 2, not 3 — Neha (67) was filtered out.

Accumulators — Compute Per Group

Inside $group: $sum · $avg · $min · $max

$sum Total / count

```
{ $sum: '$marks' }   ·   { $sum: 1 }
```

$avg Average

```
{ $avg: '$marks' }
```

$min Smallest

```
{ $min: '$marks' }
```

$max Largest

```
{ $max: '$marks' }
```

```
db.students.aggregate([
  { $group: { _id:'$city', avgMarks:{$avg:'$marks'}, maxMarks:{$max:'$marks'} } }
])
// Delhi -> avg 80.67, max 90    |    Mumbai -> avg 87.67, max 97
```

$count — Count Documents

A simple stage that outputs a single number

SYNTAX

```
{ $count: 'fieldName' }
```

Counts all documents reaching this stage. Outputs { fieldName: <count> }.

```
{ $count: 'totalStudents' }
```

```
// total students
db.students.aggregate([
  { $count: 'totalStudents' }
])

// -> { totalStudents: 15 }
```

```
// after a filter
db.students.aggregate([
  { $match:{ marks:{$gt:80} } },
  { $count: 'highScorers' }
])
// -> { highScorers: 9 }
```

$count (stage) = one number at the end · $sum:1 (accumulator) = a count per group

$project — Shape the Output

Include, exclude, and rename fields

SYNTAX

```
{ $project: { field: 1 | 0, newName: '$oldField' } }
```

1 includes, 0 excludes. Assign '$oldField' to a new name to rename.

```
{ $project: { _id:0, studentName:'$name', marks:1 } }
```

```
// include / exclude
db.students.aggregate([
  { $project: {
      _id: 0,
      name: 1,
      city: 1
  } }
])
```

```
// rename fields
db.students.aggregate([
  { $project: {
      _id: 0,
      studentName: '$name',
      location: '$city'
  } }
])
```

$sort · $limit · $skip Stages

Top-N and pagination inside the pipeline

$sort

order docs (1 / -1)

```
{ $sort: { marks: -1 } }
```

$limit

keep first n

```
{ $limit: 3 }
```

$skip

ignore first n

```
{ $skip: 2 }
```

Top 3 scorers:

```
db.students.aggregate([
  { $sort: { marks: -1 } },
  { $limit: 3 }
])  // Karan97, Tarun95, Divya93
```

3rd & 4th place (skip + limit):

```
db.students.aggregate([
  { $sort: { marks: -1 } },
  { $skip: 2 }, { $limit: 2 }
])  // Divya93, Meera91
```

⚠️ **Order matters**

$sort must come BEFORE $limit — otherwise you limit a random subset, not the true top-N.

$addFields — Add a Computed Field

Attach a new property to every document

SYNTAX

{ $addFields: { newField: <expression> } }

Adds/derives a field on every doc. Doesn't group or filter.

```
{ $addFields: { bonusMarks: { $add: ['$marks', 5] } } }
```

```
db.students.aggregate([
  { $addFields: { bonusMarks: { $add: ['$marks', 5] } } }
])
// Aarav: marks 85 -> bonusMarks 90     |     Karan: 97 -> 102
```

📌 **Don't confuse these three**

$addFields adds a field · **$add** does arithmetic · **$sum** is a group accumulator

$cond — Conditional Logic

if / then / else inside aggregation

SYNTAX

```
{ $cond: { if: <test>, then: <valTrue>, else: <valFalse> } }
```

Returns one of two values based on a condition. Used inside $project to build computed fields.

```
grade: { $cond: { if:{$gte:['$marks',90]}, then:'A', else:'B' } }
```

Nested $cond — letter grades (A / B / C):

```
db.students.aggregate([
  { $project: { _id:0, name:1, marks:1,
      grade: {
        $cond: { if: { $gte: ['$marks', 90] }, then: 'A',
          else: { $cond: { if: { $gte: ['$marks', 80] }, then: 'B',
                           else: 'C' } } }
      }
  } }
])   // Rohan 90 -> A,  Aarav 85 -> B,  Neha 67 -> C
```

$out — Save Results

Write the pipeline output to a collection

SYNTAX

```
{ $out: 'collectionName' }
```

Writes output to a collection instead of displaying it. MUST be the last stage.

```
{ $out: 'cityReport' }
```

```
db.students.aggregate([
  { $group: { _id:'$city', totalStudents:{$sum:1}, avgMarks:{$avg:'$marks'} } },
  { $out: 'cityReport' }       // save instead of display
])

db.cityReport.find()           // read the saved report any time
```

⚠️ **$out REPLACES the target collection entirely — it does NOT append. Use a fresh name unless you mean to overwrite.**

Day 4 — In a Nutshell

Refine find()

```
sort (1/-1)
limit · skip
projection
skip+limit=paging
```

$match / $group

```
filter, then group
_id = group key
use '$field'
```

Accumulators

```
$sum $avg
$min $max
$count
$sum:1 = count
```

Shape

```
$project
(rename)
$addFields
$sort/$limit/$skip
```

Logic + Save

```
$cond
(if/else)
$out (last,
replaces)
```

Golden rule: stages run top-to-bottom — $match early (speed), $sort before $limit (true top-N).

Days 1-3
CRUD · operators · querying

Day 4
Aggregation pipeline

Next ->
Indexes · Mongoose · Node/Express